2026 Spring

Course ID: 1142-1764

Foundations of Computer Science

05-OS and Cloud Computing

1

HOUR

10:10 – 11:00

Topic

Course Introduction

Cloud Computing

Course Introduction

Welcome to the course — let's get started.

Week 1

50 minutes

Course Info

Introduction to Computer Science

Fundamentals · Organization · Networks · OS · Algorithms · Data Structures · ADT

2 Semesters · 11 Topics · 32 Weeks · 3 Credits each

INSTRUCTOR	DEPARTMENT	LANGUAGE	TOOLS
Prof. H.T. Chu	**International Program in Computer Science**	**English (EMI)**	**AWS Academy**

ASSESSMENT (EACH SEMESTER)

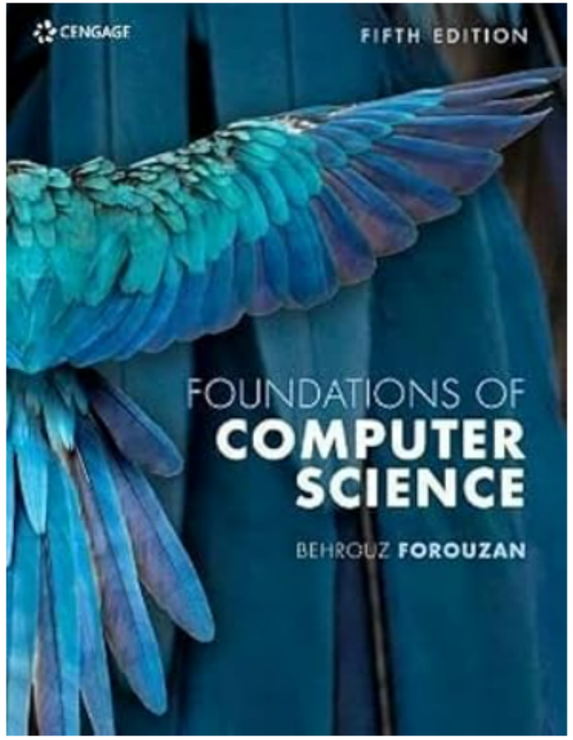
Assignments	Midterm Exam	Project / Labs	Final Exam	Participation
30%	**20%**	**30%**	**20%**	**Bonus 5%**

Providence University · Department of Computer Science and Information Engineering · AY 2025–2026

Thursday, 3-4 (10:10-12:00)

Providence Hall R307

The Textbook



Foundations of Computer Science Paperback

作者 Behrouz Forouzan (Author)

The symposium was held Oct.-Nov. 1989, Research Triangle Park, North Carolina. One hundred papers in theoretical computer science treat dispersers, deterministic amplification, and weak random sources; efficient NC algorithms for set cover with applications to learning and geometry; the inverse of automorphism in polynomial time; and speeding-up linear programming using fast matrix multiplication. Acidic paper; no subject index. Annotation copyright Book News, Inc. Portland, Or.

報告此產品或賣家的問題

語言	尺寸	ISBN-10	ISBN-13
English	18.9 x 24.7 x 2.6 cm	1473787327	978-1473787322

查看所有詳細內容

1 Introduction

Turing Model
Von Neumann Model
Computer Components
History
Social and Ethical Issues
Computer Science as a Discipline
Outline of the course
End-Chapter Materials
Practice Set

2 Number Systems

Introduction
Positional Number Systems
Non-Positional Number Systems
End-Chapter Materials
Practice Set

3 Data Storage

Data Types
Storing Numbers
Storing Text
Storing Audio
Storing Images
Storing Video
End-Chapter Materials
Practice Set

4 Operations on Data

Logic Operations
Shift Operations
Arithmetic Operations
End-Chapter Materials
Practice Set

5 Computer Organization

Introduction
Central Processing Unit
Main Memory
Input/Output Subsystem
Subsystem Interconnection
Program Execution
Different Architectures
A Simple Computer
End-Chapter Materials
Practice Set

6 Computer Networks

Introduction
TCP/IP Protocol Suite
Layers
Internet Applications
The World Wide Web (www)
End-Chapter Materials
Practice Set

7 Operating Systems

Introduction
Evolution
Components
A Survey of Operating Systems
End-Chapter Materials
Practice Set

8 Algorithms

Concept
Three constructs
Algorithm Representation
A More Formal Definition
Basic Algorithms
Subalgorithms
Recursion
End-Chapter Materials
Practice Set

9 Efficiency of Algorithms

Definition
Common Algorithms
Big-O Notation
End-Chapter Materials
Practice Set

10 Programming Languages

Evolution
Translation
Programming Paradigms
Common Concepts
End-Chapter Materials
Practice Set

11 Software Engineering

The Software Life cycle
Analysis Phase
Design Phase
Implementation Phase
Testing Phase
Documentation
End-Chapter Materials
Practice Set

12 Data Structure

Arrays
Records
Linked Lists
End-Chapter Materials
Practice Set

13 Abstract Data Types

Background
Stacks
Queues
General Linear Lists
Trees
Graphs
End-Chapter Materials
Practice Set

This semester

6 Computer Networks

Introduction
TCP/IP Protocol Suite
Layers
Internet Applications
The World Wide Web (www)
End-Chapter Materials
Practice Set

7 Operating Systems

Introduction
Evolution
Components
A Survey of Operating Systems
End-Chapter Materials
Practice Set

8 Algorithms

Concept
Three constructs
Algorithm Representation
A More Formal Definition
Basic Algorithms
Subalgorithms
Recursion
End-Chapter Materials
Practice Set

9 Efficiency of Algorithms

Definition
Common Algorithms
Big-O Notation
End-Chapter Materials

12 Data Structure

Arrays
Records
Linked Lists
End-Chapter Materials
Practice Set

13 Abstract Data Types

Background
Stacks
Queues
General Linear Lists
Trees
Graphs

+ aws academy

Operating Systems

& AWS Containers

Fundamentals of Computer Science · 電腦科學概論

College of Computing and Informatics | Providence University | 靜宜大學

Course Agenda 課程大綱

01 What is an Operating System?

02 OS Architecture & Kernel

03 Process Management

04 Memory Management

05 File Systems & I/O

06 Intro to Containers & Docker

07 Container Orchestration (K8s)

08 Amazon ECR & ECS

09 Amazon EKS & Fargate

10 Deployment & Best Practices

SECTION 01

Operating Systems

What is an OS, its role, and its evolution

What is an Operating System?

Definition

An Operating System (OS) is system software that manages computer hardware and software resources, providing common services for computer programs. It acts as an intermediary between users and hardware.

Resource Manager

Allocates CPU, memory, and I/O devices among processes

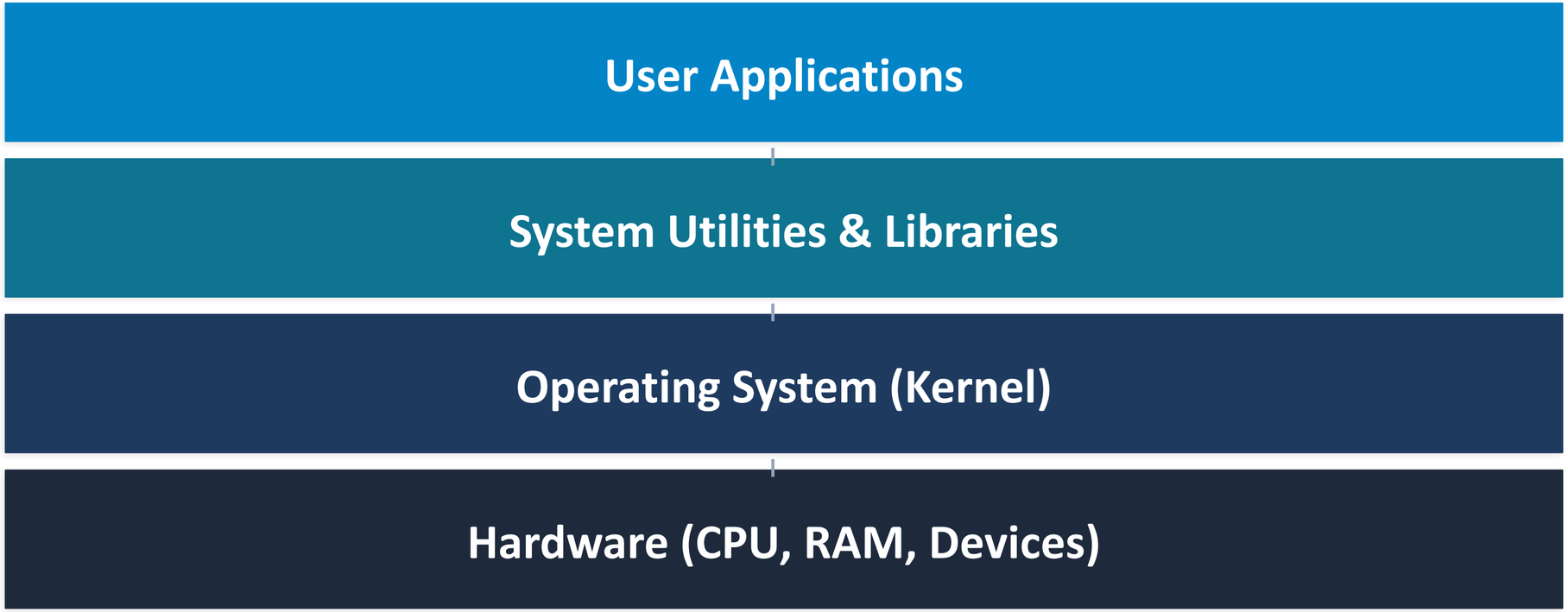
Extended Machine

Provides a virtual machine environment easier to use than bare hardware

Interface Provider

Offers API/system calls for user programs to interact with hardware

OS as a Layered Architecture



Arrows indicate abstraction: each layer uses services from the layer below and provides services to the layer above.

Key Insight: The OS shields hardware complexity from applications. Programmers write to OS APIs (system calls), not to hardware registers directly.

History of Operating Systems

1950s — **No OS**

Programs run directly on hardware. Operator manually loaded punch cards.

1960s — **Batch Systems**

Jobs queued and processed without user interaction. IBM OS/360.

1970s — **Time-Sharing & UNIX**

Multiple users share CPU. UNIX introduced in 1969 at Bell Labs.

1980s — **Personal Computers**

MS-DOS for PCs. Apple Macintosh introduces GUI. Desktop computing era.

1990s — **Windows & Linux**

Windows 95 standardizes GUI. Linux kernel created by Linus Torvalds.

2000s+ — **Mobile & Cloud**

iOS/Android dominate mobile. Cloud OS like AWS Lambda emerge.

Types of Operating Systems

Batch OS

Executes batches of jobs without user interaction. Efficient for long-running tasks. Example: IBM OS/360.

Time-Sharing OS

Allows multiple users to share CPU time simultaneously. Fast context switching creates illusion of parallelism.

Real-Time OS (RTOS)

Responds to events within guaranteed time bounds. Hard RTOS (safety-critical) vs Soft RTOS. Example: FreeRTOS.

Distributed OS

Manages multiple computers as a unified system. Resources shared transparently across network nodes.

Mobile OS

Designed for smartphones/tablets. Power-efficient, touch

Embedded OS

Lightweight OS for dedicated hardware (cameras, routers

SECTION 02

OS Architecture

Kernel design, system calls, and core components

Kernel Architecture

Monolithic Kernel

- All OS services run in kernel space as a single large program
- Fast inter-module communication via function calls
- A bug in any service can crash the entire OS
- Example: Linux, early UNIX
- High performance — used in most modern OSes
- Modules can be loaded/unloaded dynamically in Linux

Microkernel

- Only essential services (IPC, basic scheduling) in kernel space
- All other services run as user-space processes
- Better fault isolation — a crashed service doesn't crash the OS
- Examples: Mach (macOS base), QNX, seL4
- Slightly slower due to message-passing overhead
- Favored in safety-critical systems

System Calls 系統呼叫

A system call is the programmatic way an application requests a service from the operating system kernel.

Process Control

fork(), exec(), exit(), wait()

File Management

open(), read(), write(), close(), unlink()

Device Management

ioctl(), read(), write() for devices

Information Maintenance

getpid(), alarm(), sleep(), time()

Communication (IPC)

pipe(), socket(), send(), recv()

Protection & Security

chmod(), chown(), umask(), setuid()

SECTION 03

Process Management

Processes, states, scheduling, and synchronization

What is a Process? 行程

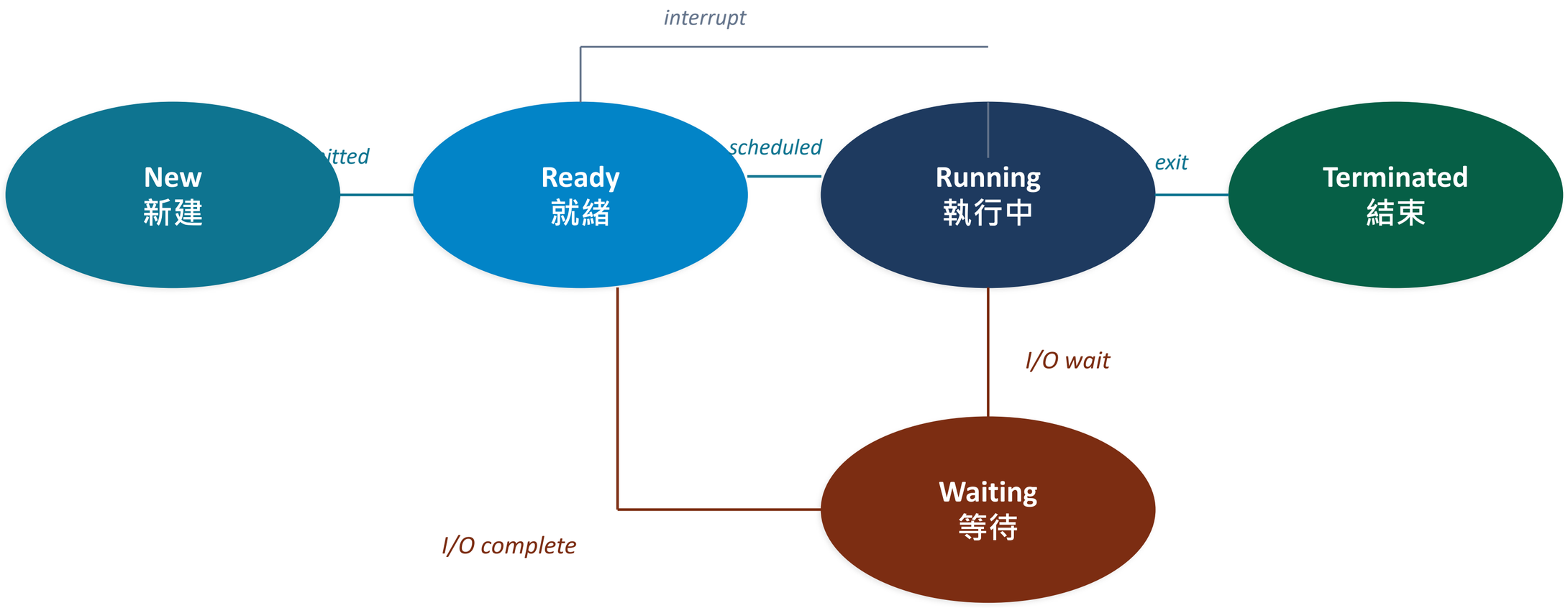
Process vs Program

- A program is passive: a file on disk (executable code)
- A process is active: a program in execution with resources
- One program can spawn multiple processes (e.g., Chrome tabs)
- Process = code + data + stack + heap + OS resources
- Each process has a unique PID (Process ID)

Process Components

- Text section: executable code
- Data section: global variables
- Heap: dynamic memory allocated at runtime
- Stack: function calls, local variables, return addresses
- Program Counter (PC): address of next instruction
- CPU registers: current activity snapshot

Process State Diagram 行程狀態



New → Admitted (process created) → Ready (waiting for CPU) → Running (executing) → either Terminated or back to Waiting/Ready

Process Control Block (PCB)

The OS stores all information about a process in a PCB (also called Task Control Block)

Process State

New, Ready, Running, Waiting, Terminated

Process ID (PID)

Unique identifier for this process

Program Counter

Address of next instruction to execute

CPU Registers

Saved register contents for context switches

Memory Limits

Base & limit registers, page table info

I/O Status

Open file list, pending I/O requests

Accounting Info

CPU time used, time limits, job numbers

Scheduling Info

Priority level, pointers in scheduling queues

CPU Scheduling 排程

- CPU Scheduling decides which process in the Ready Queue gets the CPU next
- Goals: maximize CPU utilization, minimize response time, maximize throughput, ensure fairness
- Scheduling occurs when: process terminates, process switches from Running → Waiting, process switches from Waiting → Ready, new process created
- Preemptive: OS can forcibly remove CPU from a running process (modern OSes)
- Non-preemptive (cooperative): process holds CPU until it voluntarily releases (older OSes)
- Criteria: CPU utilization (keep CPU busy), Throughput (jobs/time), Turnaround time (submission→completion), Waiting time (time in ready queue), Response time (first response)

CPU Scheduling Algorithms

FCFS – First Come First Served

Non-preemptive. Simple. Convoy effect — short jobs wait behind long ones.

SJF – Shortest Job First

Non-preemptive or preemptive (SRTF). Optimal for avg. waiting time. Hard to predict burst time.

Round Robin (RR)

Each process gets a time quantum (e.g., 10ms). Preemptive. Balances fairness and responsiveness.

Priority Scheduling

Highest priority process runs first. Can cause starvation — solved by aging (gradually raise priority).

Multilevel Queue

Multiple queues with different priorities. Foreground (RR) vs background (FCFS) processes.

Multilevel Feedback Queue

Processes can move between queues. Used in modern OSes (Linux CFS, Windows). Most flexible.

SECTION 04

Memory Management

Virtual memory, paging, segmentation, and allocation

Memory Hierarchy 記憶體層次

FASTER

CPU Registers < 1KB | ~1 ns | Most expensive

L1/L2/L3 Cache KB–MB | ~5 ns | Built into CPU die

Main Memory (RAM) 4–64 GB | ~100 ns | Volatile DRAM

SSD (Flash Storage) 100 GB–4 TB | ~0.1 ms | Non-volatile

HDD / Magnetic Disk 1 TB–20 TB | ~10 ms | Slowest persistent

LARGER

Virtual Memory 虛擬記憶體

Concept

- Illusion of more memory than physically available
- Each process has its own virtual address space
- OS + hardware (MMU) translate virtual → physical addresses
- Pages not in RAM are stored on disk (swap space)
- Page fault: accessing a virtual page not currently in RAM
- Demand paging: load pages only when needed

Benefits

- Programs larger than RAM can run successfully
- Processes are isolated from each other's memory
- Enables memory sharing between processes (shared libs)
- Simplifies memory allocation for programmers
- Enables efficient multitasking on limited hardware
- Foundation for containerization and virtual machines

Paging 分頁

Physical memory divided into fixed-size frames; virtual memory into pages of the same size (typically 4 KB). A page table maps virtual pages to physical frames.

Virtual Address Space	Page Table	Physical Memory
Page 0	→ Frame 3	Frame 0 (Page 3)
Page 1	→ Frame 1	Frame 1 (Page 1)
Page 2	→ Disk	Frame 2 (Page 4)
Page 3	→ Frame 0	Frame 3 (Page 0)
Page 4	→ Frame 2	Frame 4 (free)

Segmentation & Memory Allocation

Segmentation

- Divides memory into variable-sized logical segments: code, data, stack, heap
- Matches programmer's view of memory
- Segment Table maps segment → physical base+limit
- Allows sharing of segments between processes
- External fragmentation: free space scattered in small chunks
- Modern systems combine paging and segmentation

Memory Allocation Strategies

- First-fit: allocate first free block large enough — fast
- Best-fit: allocate smallest sufficient block — minimizes waste
- Worst-fit: allocate largest block — maximizes remaining space
- Buddy system: power-of-2 sizes, easy merging/splitting
- Slab allocator: pools of same-size objects (Linux kernel)
- Compaction: move processes to consolidate free space

SECTION 05

File Systems & I/O

Organizing persistent data and managing input/output

File System Concepts 檔案系統

- A file system organizes data stored on non-volatile storage (HDD, SSD, Flash) into a hierarchical structure
- File: named collection of related data stored on disk — has attributes: name, type, size, permissions, timestamps
- Directory (folder): container that organizes files and other directories — forms a tree structure
- Path: location of a file. Absolute: /home/user/docs/file.txt | Relative: ./docs/file.txt
- Mount point: attaching a file system to the directory tree (e.g., mounting a USB drive at /media/usb)
- Metadata: data about data — inode stores file metadata in Unix-like systems (size, owner, permissions, block pointers)
- File allocation methods: Contiguous (fast reads, fragmentation), Linked (flexible, slow random access), Indexed (inode-based, flexible and efficient)

Common File System Types

FAT32 (File Allocation Table)

Simple, widely compatible. Max file size 4 GB. Used in USB drives, SD cards. No permissions or journaling.

NTFS (Windows)

Journaling, ACL permissions, file compression, encryption, max file size 16 TB. Default for modern Windows.

ext4 (Linux)

Journaling, extents for large files, backwards compatible with ext2/3. Default for most Linux distros.

APFS (Apple)

Flash/SSD-optimized, copy-on-write, space sharing, strong encryption. Default on macOS/iOS since 2017.

ZFS / Btrfs

Advanced: built-in RAID, snapshots, checksums

tmpfs / procfs

Virtual file systems in RAM. /proc exposes kernel/process

File Permissions & Directory Structure

Unix Permission Model

- 3 categories: Owner, Group, Others
- 3 permission bits: Read (r=4), Write (w=2), Execute (x=1)
- Example: rwxr-xr-- = 754
- chmod 755 file.txt — sets permissions numerically
- chown user:group file — changes ownership
- Special bits: setuid (s), setgid (s), sticky bit (t)

Linux Directory Hierarchy

- / — root of the entire filesystem tree
- /bin — essential user binaries (ls, cat, cp)
- /etc — system configuration files
- /home — user home directories
- /var — variable data: logs, spool files
- /proc — virtual FS: process and kernel info
- /dev — device files (disks, terminals)

I/O Management & Device Drivers

Programmed I/O

CPU polls device status register continuously. Simple but wastes CPU cycles (busy waiting).

Interrupt-Driven I/O

Device sends interrupt when ready. CPU handles other work in between. More efficient than polling.

DMA (Direct Memory Access)

Special hardware controller transfers data directly between device and memory without CPU per-byte involvement. CPU only notified when done.

Device Drivers

Software module that translates generic OS I/O commands into device-specific commands. Vendor-supplied or open source.

I/O Scheduler

Reorders disk I/O requests to minimize seek time. Algorithms: FCFS, SSTF, SCAN (elevator), C-SCAN.

Buffering & Caching

Buffers smooth speed differences between producer/consumer. OS page cache keeps recently read disk blocks in RAM for fast re-access.

OS Security & Deadlocks

OS Security Mechanisms

- User authentication: passwords, MFA, biometrics
- Access Control Lists (ACL): per-resource permission matrix
- Mandatory Access Control (MAC): labels on objects (SELinux)
- Address Space Layout Randomization (ASLR): prevents exploits
- Stack canaries: detect stack buffer overflows
- Secure Boot: verify bootloader/kernel signatures

Deadlocks 死結

- Deadlock: two or more processes wait for resources held by each other — indefinitely blocked
- 4 conditions (Coffman): Mutual exclusion, Hold & wait, No preemption, Circular wait
- Prevention: eliminate one of the 4 conditions
- Avoidance: Banker's Algorithm — safe state analysis
- Detection & Recovery: kill a process or preempt resources
- Ignore (Ostrich Algorithm): used in most OSes

Popular Operating Systems

Windows 11

Desktop/Enterprise leader

NTFS, WSL2, Hyper-V, Active Directory, .NET, WinAPI, extensive hardware support, GUI-first

Linux (Kernel)

Servers, Cloud, HPC, Android

Open source, monolithic+modules, ext4/btrfs, bash CLI, Docker-native, dominates cloud servers (AWS, GCP, Azure VMs)

macOS Sonoma

Creative professionals

BSD/Mach hybrid kernel (XNU), APFS, Gatekeeper, Secure Enclave, Unix CLI, M-series silicon

Android 14

#1 mobile OS worldwide

Linux kernel base, ART runtime, sandbox per app, Binder IPC, biometric auth, ARM-optimized

Part 1 Summary — Operating Systems

Key Takeaways

OS Role	Acts as resource manager and provides abstraction layer between hardware and applications
Process Mgmt	OS tracks all processes via PCBs; scheduling algorithms decide CPU allocation order
Memory	Virtual memory + paging enables each process to have an isolated address space larger than physical RAM
File Systems	Hierarchical organization of persistent data with permissions, metadata, and allocation tables
I/O	DMA and interrupt-driven I/O let CPU work on other tasks while devices transfer data independently
Security	Deadlocks, access control, ASLR, and secure boot are fundamental OS safety mechanisms

SECTION 06

Containers & Virtualization

From virtual machines to lightweight containers

Virtualization 虛擬化

Hypervisor / VMs

- Hypervisor sits on host OS or bare metal
- Type 1 (bare metal): VMware ESXi, Hyper-V, KVM
- Type 2 (hosted): VirtualBox, VMware Workstation
- Each VM has full OS + virtual hardware
- Strong isolation — VM escape is rare
- Heavy: minutes to boot, GB RAM overhead per VM
- Great for running different OS on same hardware

Containers

- Share host OS kernel — no full OS per container
- Namespace isolation: PID, network, mount, UTS, IPC
- cgroups (control groups): limit CPU, memory, I/O
- Much lighter: seconds to start, MB overhead
- Image-based: repeatable, portable, version-controlled
- Docker, Podman, LXC are common runtimes
- Trade-off: weaker isolation vs VMs

VMs vs Containers — Side by Side

Feature	Virtual Machines	Containers
Boot Time	Minutes (full OS boot)	Seconds (process start)
Size	GBs per VM image	MBs per container image
Isolation	Strong (hardware-level)	Moderate (OS namespace)
OS Overhead	Full OS per VM	Shared host kernel
Portability	Hypervisor-dependent	Run anywhere Docker runs
Density	10s of VMs per host	100s of containers per host
Use Case	Lift-and-shift legacy apps	Microservices, CI/CD pipelines
Security	Stronger boundaries	More attack surface (shared kernel)

Docker — The Container Platform

Docker is an open platform for developing, shipping, and running applications in containers. Released in 2013, it popularized containerization for mainstream software development.

Docker Image

Read-only template containing app code, runtime, libraries, env variables. Built from a Dockerfile. Immutable snapshot.

Docker Container

A running instance of an image. Writable layer on top of image. Isolated but lightweight. Can be stopped, started, moved.

Dockerfile

Text file with instructions to build an image. FROM, RUN, COPY, CMD, EXPOSE directives. Reproducible builds.

Docker Registry

Repository for storing and distributing images. Docker Hub is the public default. Private: AWS ECR, GitHub Container Registry.

Docker Architecture 架構

Docker Client

docker build

docker pull

docker run

docker push

docker ps

Docker Daemon (dockerd)

Manages objects:

- *Images*
- *Containers*
- *Networks*
- *Volumes*

Listens on Unix socket /var/run/docker.sock

Registry

Docker Hub

AWS ECR

GitHub GHCR

Azure ACR

Private repo

Dockerfile & Key Docker Commands

Dockerfile

```
# Base image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy dependencies
COPY package*.json ./
RUN npm install

# Copy source code
COPY . .

# Expose port
EXPOSE 3000

# Start command
CMD ["node", "server.js"]
```

Essential Docker Commands

`docker build -t app:v1 .`	Build image from Dockerfile
`docker run -d -p 8080:3000 app:v1`	Run container in background
`docker ps`	List running containers
`docker logs <id>`	View container logs
`docker exec -it <id> sh`	Shell into container
`docker stop <id>`	Stop container gracefully
`docker rm <id>`	Remove stopped container
`docker images`	List local images
`docker rmi <image>`	Delete image
`docker push registry/app:v1`	Push image to registry

SECTION 07

Container Orchestration

Kubernetes and managing containers at scale

Why Container Orchestration?

When you have dozens or hundreds of containers across multiple servers, manual management becomes impossible.

Problem Without Orchestration	Orchestration Solution (K8s)
✗ Container crashes	✓ Auto-restart and self-healing
✗ Traffic spike	✓ Auto-scale replicas up/down
✗ Server failure	✓ Reschedule containers to healthy nodes
✗ Rolling updates needed	✓ Zero-downtime deployments
✗ Service discovery	✓ Built-in DNS and load balancing
✗ Secret management	✓ Encrypted secrets and config maps

Kubernetes (K8s) Overview 容器編排

- Kubernetes is an open-source container orchestration platform originally developed by Google (Borg system), donated to CNCF in 2014
- K8s = 8 letters between K and s — hence the numeronym
- Declarative model: you describe the desired state; K8s continuously works to achieve and maintain it
- Key capabilities: automated rollouts/rollbacks, service discovery, load balancing, storage orchestration, self-healing, secret management
- Cluster: a set of nodes (VMs or physical machines). Master nodes = Control Plane. Worker nodes = run workloads
- Everything in K8s is an object described in YAML: Pods, Deployments, Services, ConfigMaps, Secrets, Namespaces
- CNCF ecosystem: Helm (package manager), Istio (service mesh), Prometheus (monitoring), ArgoCD (GitOps)

Kubernetes Architecture

Control Plane (Master Node)

API Server

Single point of contact for all K8s API calls. kubectl talks here.

etcd

Distributed key-value store. Cluster state and config stored here.

Scheduler

Watches for unscheduled Pods, assigns them to nodes based on resources.

Controller Mgr

Runs controllers: ReplicaSet, Node, Job, Endpoint controllers.

Worker Nodes

kubelet

Agent on each node. Ensures containers in Pods are running and healthy.

kube-proxy

Maintains network rules on nodes. Enables service discovery and load balancing.

Container Runtime

Runs containers. Docker, containerd, or CRI-O. Downloads images, starts/stops containers.

Pods

Smallest deployable unit. One or more containers sharing network and storage.

Kubernetes Key Objects

Pod

Smallest unit. 1+ containers sharing localhost network and volumes. Ephemeral — deleted and replaced, not updated.

Deployment

Manages ReplicaSets. Declares desired pod replicas. Handles rolling updates and rollbacks. Most common object.

Service

Stable network endpoint for pods. Types: ClusterIP (internal), NodePort (external), LoadBalancer (cloud LB).

ConfigMap

Stores non-sensitive configuration as key-value pairs. Injected as env vars or files into pods.

Secret

Like ConfigMap but for sensitive data (passwords, tokens)

Ingress

HTTP/HTTPS routing to services from outside the cluster

SECTION 08

AWS Container Services

ECR, ECS, EKS, and Fargate on Amazon Web Services

AWS Container Services Overview

AWS provides a complete container ecosystem — from image storage to fully managed orchestration:

Amazon ECR	Amazon ECS	Amazon EKS	AWS Fargate
Elastic Container Registry	**Elastic Container Service**	**Elastic Kubernetes Service**	**Serverless container compute**
Store & distribute images	AWS-native container orchestration	Managed Kubernetes on AWS	No server management needed
Push/pull images Private registries Vulnerability scans	Task definitions Service auto-scaling AWS-native	Standard K8s API Managed control plane Community ecosystem	No EC2 needed Pay per vCPU/RAM Works with ECS & EKS

Amazon ECR — Elastic Container Registry

What is ECR?

- Fully managed Docker container registry on AWS
- Private by default: IAM-controlled access per repository
- Integrated with ECS, EKS, CodeBuild, Lambda
- Supports both Docker images and OCI artifacts
- Lifecycle policies: auto-delete old/untagged images
- Replication: cross-region or cross-account

Key Features

- Image scanning: Snyk/Amazon Inspector finds CVEs on push
- Immutable tags: prevent overwriting existing image tags
- Pull through cache: proxy to Docker Hub, ECR Public
- ECR Public Gallery: share images publicly (like Docker Hub)
- KMS encryption at rest for all image layers
- Fine-grained IAM: repo-level resource policies

Amazon ECS — Elastic Container Service

ECS is AWS's own container orchestration service — simpler than Kubernetes but deeply integrated with AWS services.

Cluster

Logical grouping of EC2 instances or Fargate capacity to run containers.

Task Definition

Blueprint for your containers — like a docker-compose.yml. Specifies image, CPU, memory, ports, environment, volumes.

Task

A running instance of a Task Definition. One-off jobs run as Tasks.

Service

Runs and maintains a desired number of Task replicas. Integrates with ALB for load balancing.

Container Agent

Runs on each EC2 node in the cluster. Reports resource usage and manages container lifecycle for ECS.

Service Discovery

AWS Cloud Map integration allows containers to find each other by DNS name automatically.

ECS Task Definition — Example

task-definition.json

```
{
  "family": "web-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "containerDefinitions": [{
    "name": "web",
    "image": "123.dkr.ecr.us-east-1
             .amazonaws.com/web:latest",
    "portMappings": [{
      "containerPort": 3000,
      "protocol": "tcp"
    }],
    "environment": [{
      "name": "NODE_ENV",
      "value": "production"
    }],
    "logConfiguration": {
      "logDriver": "awslogs"
    }
  }]
}
```

Family & Network

Versioned name + awsvpc networking mode

Compatibility

FARGATE or EC2 launch type

Resources

CPU units (256=0.25 vCPU) & memory MB

Port Mapping

Container port exposed on the task

Logging

CloudWatch Logs via awslogs driver

ECS Launch Types 部署方式

EC2 Launch Type

- You manage a fleet of EC2 instances in the cluster
- Full control over instance types, custom AMIs, placement
- ECS Agent runs on each EC2 instance
- Pay for EC2 instances even when idle
- Good for: workloads needing GPU, special hardware, or persistent storage
- Requires: patching, scaling, capacity planning

Fargate Launch Type

- Serverless — no servers to manage or provision
- AWS manages the underlying infrastructure
- Pay only for vCPU and memory used by running tasks
- Better security: each task gets its own kernel (micro-VM)
- Faster to get started — no cluster capacity planning
- Good for: batch jobs, microservices, variable workloads

Amazon EKS — Elastic Kubernetes Service

Overview

- Fully managed Kubernetes control plane on AWS
- AWS manages, updates, and patches the control plane
- Standard Kubernetes API — all kubectl commands work
- Integration: ALB Ingress Controller, EBS/EFS CSI, IAM OIDC
- Multi-AZ control plane: highly available by default
- Supports both EC2 and Fargate node types

EKS vs Self-Managed K8s

- No need to install/operate etcd or API Server
- Automated K8s version upgrades with minimal downtime
- Native VPC networking with Amazon VPC CNI plugin
- Built-in CloudWatch and CloudTrail integration
- AWS Shield DDoS protection for control plane
- Trade-off: slightly less flexibility vs complete control

AWS Fargate — Serverless Containers

Fargate is a serverless compute engine for containers — you define resource requirements, AWS runs the infrastructure.

1. Define Task: Set vCPU (0.25–16) and memory (0.5–120 GB) in Task Definition
2. Deploy Service: ECS or EKS Fargate profile pulls container image from ECR
3. Isolated VM: Fargate provisions a dedicated micro-VM for each task (gVisor/Firecracker)
4. Task Runs: Container executes in isolated network namespace with own elastic network interface
5. Auto-cleanup: Task completes, infrastructure is automatically released — zero idle cost

ECS vs EKS vs Fargate — Comparison

Dimension	ECS (EC2)	ECS (Fargate)	EKS (EC2)	EKS (Fargate)
Server Mgmt	You manage	None	You manage	None
K8s API	No	No	Yes	Yes
Complexity	Low	Lowest	High	Medium
Flexibility	High	Medium	Highest	High
Cost model	EC2 + ECS	vCPU+RAM	EC2 + EKS	vCPU+RAM
Best for	AWS-native apps	Serverless jobs	K8s workloads	K8s + serverless

SECTION 09

Deployment & Best Practices

CI/CD, security, networking, and cost optimization

Deploying Containers on AWS — CI/CD Pipeline

1. Code

Developer pushes code to GitHub/CodeCommit

2. Build

AWS CodeBuild builds Docker image from Dockerfile

3. Scan

ECR Image Scanning checks for CVE vulnerabilities

4. Push

Image pushed to Amazon ECR private registry

5. Deploy

ECS/EKS pulls new image, performs rolling update

6. Monitor

CloudWatch collects metrics, logs, and alarms

Container Security on AWS

Image Security

- Use minimal base images (Alpine, Distroless)
- Never run containers as root — use USER in Dockerfile
- ECR image scanning on push — fail CI if HIGH CVE found
- Sign images with AWS Signer or Cosign (supply chain security)
- Immutable image tags in production — no :latest
- Review and audit all third-party base images

Runtime & Network Security

- IAM Task Roles: each task gets least-privilege IAM role
- Secrets Manager / Parameter Store instead of env vars
- Security Groups: restrict inbound/outbound container traffic
- VPC isolation: containers in private subnets, no public IP
- AWS WAF + Shield on Application Load Balancer
- CloudTrail + GuardDuty for audit and anomaly detection

Container Networking on AWS

VPC CNI (EKS)

Each pod gets a real VPC IP address. No overlay network. Direct ENI attachment. High performance, native AWS routing.

awsvpc (ECS)

Each Fargate task/EC2 task gets its own Elastic Network Interface (ENI) and private IP in your VPC. Full security group support.

App Load Balancer

ALB routes HTTP/HTTPS to ECS services. Path-based and host-based routing. Integrates with ACM for TLS. Target group per service.

Service Discovery

AWS Cloud Map: containers register with Route 53 private hosted zones. Services find each other via DNS: api.myapp.local.

Service Mesh

AWS App Mesh (Envoy-based) or Istio on EKS. mTLS between services, circuit breakers, traffic shifting, observability.

PrivateLink

Access AWS services (ECR, S3, Secrets Manager) from containers without internet gateway. Stays within AWS backbone network.

Container Cost Optimization on AWS

Right-Sizing

- Use AWS Compute Optimizer to analyze task CPU/memory usage
- Start small and scale up — over-provisioning wastes money
- Use Graviton (ARM) instances — up to 40% cheaper, better perf
- Fargate Spot: up to 70% savings for fault-tolerant tasks
- Reserved capacity (Savings Plans) for steady-state workloads
- Multi-AZ spot with diversified instance types for resilience

Architecture Best Practices

- Auto-scaling: use ECS Service Auto Scaling with CloudWatch metrics
- Scale to zero: use Fargate for sporadic workloads — no idle cost
- Optimize Docker images: multi-stage builds, layer caching
- Use ECR lifecycle policies to auto-delete old images
- S3 + CloudFront for static assets instead of containers
- Monitor costs with AWS Cost Explorer and container-level tags

Monitoring & Observability for Containers

- The three pillars: Metrics (what happened), Logs (what was said), Traces (what path was taken)
- Amazon CloudWatch Container Insights: collect CPU, memory, network, disk metrics per container/task/service
- CloudWatch Logs: all ECS/EKS container stdout/stderr → awslogs log driver → CloudWatch Log Groups
- AWS X-Ray: distributed tracing. Trace requests across microservices. Visualize service maps and latency hotspots
- Prometheus + Grafana on EKS: industry standard for K8s monitoring. AWS Managed Prometheus + Managed Grafana
- Health checks: configure ECS/K8s liveness and readiness probes to auto-replace unhealthy containers
- Alerts: CloudWatch Alarms on container metrics → SNS → PagerDuty / Slack for on-call notifications

Container Use Cases in Industry

Microservices Architecture

Netflix, Uber: 100s of services each in separate containers. Independent deploy, scale, and fault isolation per service.

CI/CD Build Agents

AWS CodeBuild/GitHub Actions spin up containers per job. Clean environment every build. No shared state between runs.

Machine Learning

SageMaker uses containers for training jobs. Custom Docker image with PyTorch/TensorFlow. Reproducible ML pipelines.

Batch Processing

AWS Batch on ECS/Fargate: process S3 data, transcode video. Pay per second. Thousands of parallel tasks.

API Backend

Containerized Node.js/Python APIs behind ALB on ECS

Edge / IoT

AWS Greengrass deploys containers to edge devices. Local

Connecting the Dots: OS ↔ Containers

Containers depend on OS kernel features — understanding the OS explains WHY containers work the way they do.

OS: Namespaces (process isolation)	Container: Each container has isolated PID, network, filesystem, and user namespace — appears to be its own machine
OS: cgroups (resource limits)	Container: Docker/K8s set CPU and memory cgroup limits on containers — the same knobs OS uses to manage processes
OS: Union File System (overlay FS)	Container: Container image layers are overlayFS mounts — OS kernel feature enabling efficient copy-on-write layers
OS: System Calls + seccomp	Container: Containers share kernel syscalls with host. Seccomp profiles whitelist allowed syscalls for security
OS: Network namespaces + iptables	Container: Container networking uses Linux veth pairs, iptables NAT rules, and network namespaces — all OS features

Hands-On Lab Ideas 實作練習

Lab 1

Explore Process Scheduling

Run top/htop. Use nice/renice to change process priority. Write a CPU-bound Python script and observe scheduler behavior.

Lab 2

File Permissions & System Calls

Use chmod, chown, ls -la. Use strace to trace syscalls made by a simple program. Try running ls, cat.

Lab 3

Docker Hello World → WebApp

Install Docker. Build a Dockerfile for a Node.js app. Run locally on port 8080. Push to Docker Hub.

Lab 4

Deploy to ECS Fargate

Create ECR repo, push image. Create ECS cluster, Task Definition, Service. View logs in CloudWatch.

Quick Reference: Key Concepts & Commands

Operating Systems	
Kernel	Core OS: manages HW, syscalls
Process	Running program with resources
PCB	OS data structure per process
Virtual Memory	Illusion of large address space
Page Fault	Accessing page not in RAM
Deadlock	Circular wait for resources
System Call	User→Kernel mode API

Containers & AWS	
Image	Immutable container template
Container	Running instance of image
Pod (K8s)	Smallest K8s deployable unit
ECR	AWS image registry service
ECS	AWS container orchestration
EKS	Managed Kubernetes on AWS
Fargate	Serverless container compute

課程總結與後續學習方向

OS = bridge between hardware and software. Manages processes, memory, files, I/O, and security.

Process scheduling, virtual memory, and file systems are foundational to all modern computing.

Containers use OS namespaces + cgroups to provide lightweight, isolated application environments.

Docker standardized container build-ship-run workflow. Images → ECR. Run on ECS, EKS, or Fargate.

Next: System Security | Cloud Architecture | DevOps & CI/CD | Distributed Systems